

Economic Cycles

Launch Plan for an Economics & Investment Education App

From working prototype to app stores — a complete playbook for a first-time builder

Prepared for WJ · August 2026

Contents

1. Executive Summary and Reality Check

You already have more than most people who say they want to build an app: a working v5 prototype with 12 structured lessons based on Ray Dalio's economic-machine framework, five languages, a quiz engine, a kids section, a glossary, and a market dashboard. The job now is not to invent a product — it is to turn a prototype into something installable, sellable, and improvable by one person with no development background, on a budget of roughly \$50 per month.

This plan makes three strategic calls up front, and it is honest about a tension in your own choices. You chose to launch on web, iOS, and Android from day one. That is achievable with a single codebase (Expo), but 'day one' should not mean 'the same day.' The plan therefore sequences it: one codebase from the start, web released first (weeks 1–8), app stores four to six weeks later. You lose nothing — the code is shared — and you gain a place to make mistakes cheaply before Apple's reviewers and public app-store ratings are watching.

The three calls: First, build once with Expo (React Native) so web, iOS, and Android come from the same code — your prototype is already React, so most of it carries over. Second, sell structure and habit, not information. People will not pay for facts they can get from ChatGPT; they will pay for a guided path, streaks, progress, and the feeling of getting somewhere — the Duolingo lesson, applied to economics. Third, run the project as a system that audits itself: every month you will run a blindspot review against a checklist in this document, and every feature ships with a number that can prove it wrong.

RECOMMENDED ACTION: Read Section 9 (the self-refuting system) and Section 10 (blindspot register) before writing any code. They change what you build first.

2. Product Architecture and Tech Stack

2.1 The stack, in plain language

Think of the app as four layers. The app itself is what users touch — screens, lessons, animations. The backend is a service that remembers things (accounts, lesson progress, quiz scores) so a user who switches phones keeps their streak. The payment layer handles subscriptions across Apple, Google, and the web without you writing billing code. The analytics layer records what users actually do, which is the fuel for the self-improving process in Section 9.

Layer	Tool	Why this one	Cost
App (all 3 platforms)	Expo (React Native) + Expo Router	One codebase for web, iOS, Android. Your v5 prototype is React, so lesson logic and content transfer with modest rework.	Free tier: 15 iOS + 15 Android builds/mo — enough for a solo builder
Backend	Supabase	Accounts, database, and progress sync with almost no server code. Generous free tier.	\$0 until real scale
Payments	RevenueCat	One integration covers App Store, Play Store,	Free below \$2.5k/mo

		and Stripe on web; handles receipts and trials.	revenue
Ads (free tier only)	Google AdMob	Largest fill rate for a small app; rewarded video fits education well.	Free (they pay you)
Analytics	PostHog	Event tracking, funnels, session replays. Free tier is ample at launch.	\$0 at launch scale
Web hosting	Vercel or EAS Hosting	Deploys the web build on every update.	\$0

2.2 Restructuring your prototype

Your v5 file is 1,340 lines in a single .jsx file: translations, lesson content, quiz data, and all UI components together. That was the right way to sketch; it is the wrong way to grow. The single most important refactor is separating content from code. Lessons, quiz questions, glossary terms, and translations become data files (JSON); the app becomes a machine that renders whatever data it is given. After this split, adding lesson 13 or fixing a Korean translation never risks breaking the app — and later, content can live in Supabase so you can update lessons without an app-store release.

Target structure: an app/ folder with one file per screen (the four tabs plus lesson, quiz, and payoff screens); a content/ folder with lessons.json, quizzes.json, glossary.json; a locales/ folder with one file per language (en.json, ko.json, es.json, zh.json, and your fifth); a components/ folder for reusable pieces (LessonCard, CycleDiagram, ProgressRing); and a lib/ folder for logic (progress tracking, purchases, analytics). Ask your AI coding tool to perform this migration file by file — it is mechanical work AI does well.

2.3 The Markets tab problem

Your dashboard hardcodes 'February 2026.' A finance app showing stale data loses all credibility instantly — worse than showing none. Two honest options: make the tab explicitly educational ('here is what an inverted yield curve looks like and what it has historically meant') with no dates, or wire it to FRED, the Federal Reserve's free public data API, for real yield-curve and rate data. Start with the first — it is zero-risk and zero-cost — and add FRED data as a post-launch premium feature.

RECOMMENDED ACTION: Refactor before adding anything new: split content into JSON, screens into files. Then initialize an Expo project and move the pieces in. This is weeks 1–2 and it is the foundation everything else stands on.

3. UI/UX Design

3.1 What to keep

Your four-tab layout (Home, Learn, Markets, More) is standard mobile navigation — keep it. Sequential lesson unlocking, per-lesson 'Key Takeaway' and 'Think About This' sections, and the age-banded kids content are all sound structures. The redesign work is polish and psychology, not rearrangement.

3.2 The first five minutes decide everything

Most education apps lose the majority of installers in the first session. Design the first five minutes as your most important feature. On first open, skip registration entirely — let users into lesson 1 immediately and ask them to create an account only when there is something worth saving (their progress, at the end of lesson 1). Ask one or two light onboarding questions ('What describes you best: total beginner / know some basics / active investor?') and use the answer to set tone, not to gate content. Lesson 1 should take under four minutes and end with a small win: a completed-lesson animation, a progress ring at 1/12, and a one-tap 'Continue tomorrow' streak prompt.

3.3 Habit mechanics

Borrow the mechanics that make Duolingo work, without the guilt-trip excess: a visible streak counter on Home; a progress ring showing lessons completed; an estimated 'minutes to finish today's lesson' label (people commit to five minutes, not to 'learning economics'); a daily reminder notification the user opts into, worded as curiosity ('Why do recessions actually start? Lesson 5 is ready') rather than nagging. Each quiz should give immediate, kind feedback — your existing quizExplain pattern is right.

3.4 Visual system

Pick one accent color per cycle phase (you already do this with per-lesson colors — good), one neutral background system with dark mode from day one (finance audiences skew toward dark mode), and one typeface (Inter is free and excellent). Design in Figma's free tier only for screens you are unsure about; for the rest, iterate directly in code with your AI tool — faster for a solo builder. Animated diagrams are your differentiator: an LLM can explain the yield curve in text, but your animated curve inverting in front of the user is something a chat window does not do. Invest polish there.

3.5 Accessibility and languages

Ship English polished at launch. Keep the other four languages in the app — they cost little since the i18n structure exists — but mark them 'beta' and do not market them until each is reviewed by a native speaker. Support dynamic font sizes and screen-reader labels on interactive elements; app stores increasingly check this, and it widens your audience at near-zero cost.

RECOMMENDED ACTION: Storyboard the first-session flow (open → lesson 1 → win → account prompt → streak opt-in) on paper before coding it. Then build that flow end-to-end before any other screen work.

4. Pricing Plan

4.1 Answering your own objection

You are right that nobody pays a premium for information alone — economics facts are free online and every LLM explains them. But look at what the market actually pays for: Bloom charges \$15/month or \$120/year to teach teens investing; Finimize charges about \$200/year for market briefings. What people buy is curation (someone decided what matters and in what order), progression (a path with a visible end), habit (the app makes them do it), and identity ('I'm someone who understands the economy'). Your price must be set for that — real, but modest.

4.2 The tiers

Tier	Price	What's included
Free	\$0 (with ads)	Lessons 1–4, basic quiz, glossary, Markets tab. Rewarded ads only. The free tier is your marketing: it must be good enough to recommend.
Premium monthly	\$6.99/mo, 7-day free trial	All 12+ lessons, all quizzes with explanations, kids section, no ads, offline access, future live-data features.
Premium yearly	\$39.99/yr (≈\$3.33/mo)	Same as monthly. Priced so yearly is the obvious choice — this is where most revenue should come from.
Founding lifetime	\$79.99 one-time, first 500 buyers	Everything forever plus a founder badge. Generates early cash and creates invested users who give feedback.

This deliberately undercuts Bloom by more than half and Finimize by 80%. As an unknown solo product you win on price and focus, not brand. Two fee facts work in your favor: Apple's Small Business Program cuts its commission from 30% to 15% for developers under \$1M/year (you must apply for it — it is not automatic), and Google has an equivalent program. On web, Stripe takes roughly 3%, which is why the web version should be where you send people to subscribe whenever possible.

4.3 Where the paywall sits

Gate at the moment of maximum motivation: the user finishes lesson 4 (free tier ends) right as the story reaches long-term debt cycles — the part people most want to understand. Show the paywall with a preview of what lessons 5–12 answer, defaulting to the yearly plan with the trial. Never paywall mid-lesson, and never interrupt a first session with an upsell.

4.4 What to expect, numerically

Freemium education apps typically convert 1–5% of active users to paid. Be conservative: at 2% conversion, you need roughly 5,000 monthly active users to reach 100 subscribers — about \$350/month

if they split between monthly and yearly. That is the realistic year-one shape of this business: a slope, not a spike. The pricing is designed so the business works at small numbers and scales without change.

RECOMMENDED ACTION: Launch with exactly these four tiers and change nothing for 90 days. Pricing experiments before you have ~2,000 users produce noise, not signal. Apply to both small-business fee programs the week your store accounts are approved.

5. How to Draw Users — and Make Them Pay

5.1 The content flywheel (your main engine)

With no ad budget, your acquisition engine is content made from the app itself. Your animated economic diagrams are inherently short-form-video material: a 30-second clip of the yield curve inverting with a voiceover ('this shape has preceded most US recessions — here's why') is exactly what performs on TikTok, YouTube Shorts, and Instagram Reels. Every lesson yields two or three such clips. Post three per week; each ends with the app name. This costs time, not money, and it compounds: the clips keep pulling installs for months. Screen-record the app itself for the visuals — the content pipeline and the product are the same thing.

5.2 Launch moments

Stack the free launch channels: Product Hunt (the web version launches here — prepare a good demo GIF and launch on a Tuesday–Thursday), Hacker News Show HN (the Dalio-machine angle plus interactive animations is genuinely Show-HN-worthy), and Reddit communities like r/personalfinance, r/investing, and r/Bogleheads — but read each subreddit's self-promotion rules first, and lead with a useful animated explainer, not a download link. One good Reddit post that teaches something can outperform a month of ads.

5.3 App Store Optimization (ASO)

Most durable installs for education apps come from store search. Target phrases people actually type: 'learn investing,' 'economics for beginners,' 'how the economy works,' 'interest rates explained.' Your title and subtitle should contain the two strongest phrases. Screenshots should show the animated diagrams and lesson flow, not the home screen. From day one, prompt happy users for ratings at a moment of success (right after a completed quiz with a good score) — early rating volume is disproportionately important.

5.4 The web version as top-of-funnel

Make lessons 1–2 playable on the web without any signup and make each lesson a shareable URL. Every social clip links there — zero-friction taste of the product — and the page's job is to convert visitors to app installs or accounts. This is the practical payoff of your 'all platforms' decision: web for discovery and cheap payments, apps for retention and habit.

5.5 Converting free users to paying ones

Conversion levers in order of power: the lesson-4 paywall at peak curiosity (Section 4.3); the 7-day yearly trial (trials convert far better than hard paywalls); a well-timed email — collect emails at account creation and send a short 'what you'll learn in lessons 5–12' note two days after someone stalls at the paywall; and an annual-plan discount offered once, when a monthly subscriber hits 60 days. Do not add referral programs, coupon codes, or seasonal sales in year one — each adds complexity that a solo founder pays for in maintenance.

RECOMMENDED ACTION: Before launch, produce ten short clips from app screen recordings and schedule them. Launch order: web on Product Hunt + Show HN first, then app stores once the web version has survived contact with real users.

6. In-App Ads: A Deliberately Small Role

Ads belong in this product only as a gentle tax on the free tier and a nudge toward Premium — not as a revenue pillar. The arithmetic explains why. Ad revenue is measured in eCPM (earnings per 1,000 ad views). Rewarded video — where a user chooses to watch an ad to earn something — pays best: roughly \$8–18 eCPM for a global audience, and up to \$18–45 in top-tier markets, with finance audiences often earning a 2–3× premium because advertisers value them. Even so: 1,000 free users each viewing one rewarded ad a day yields very roughly \$240–540 a month at global rates. Meaningful, but 100 yearly subscribers beat it — and ads degrade the experience that creates subscribers.

Format	Typical global eCPM	Use in this app?
Rewarded video (user opts in)	\$8–18	Yes — the only format at launch. 'Watch a short ad to unlock tomorrow's lesson today' or to retry a failed quiz.
Interstitial (full-screen between actions)	\$2.50–5	Sparingly, later, free tier only — at most one per session, between lessons, never mid-lesson.
Banner	under \$1	No. Clutters a learning UI for pennies.

Three rules. Premium users never see any ad, ever — it is half the reason to pay. Do not integrate AdMob until you have roughly 1,000 daily active users; below that, revenue is coffee money and the SDK adds complexity and privacy-disclosure burden. And critically: never serve ads in the kids section (Section 10 explains the legal reason).

RECOMMENDED ACTION: Launch with zero ads. Add rewarded video to the free tier only after crossing ~1,000 DAU, and measure whether ad-exposed users convert to Premium at a different rate — that number tells you whether ads are helping or hurting.

7. Your Development Workflow (No Experience Required)

7.1 The toolkit

You will not be writing most of the code yourself — you will be directing an AI that writes it, and your real job is deciding what to build, testing it, and asking good questions. The toolset: Cursor (an AI-first code editor, ~\$20/month) or Claude Code (AI coding in a terminal) as your builder — pick one, and Cursor is the friendlier start for a beginner because you can see files visually; GitHub (free) as the versioned home of your code — every change is saved, and any mistake can be rolled back, which removes most of the fear of breaking things; Expo Go (free app on your phone) to see your app live on a real device while developing; and EAS, Expo's build service, to produce the actual store binaries — its free tier of 15 builds per platform per month is plenty.

7.2 A working rhythm

Work in small loops: describe one change to the AI in plain language ('when a lesson is completed, show a celebration animation and update the progress ring'), let it implement, test on your phone, commit to GitHub with a one-line note. Never batch ten changes before testing — when something breaks you won't know which change did it. Keep two versions of the app: a development version only you see, and the released version users have. Fixed weekly rhythm: build Monday–Thursday, release to the web Friday, submit store updates every two to three weeks (store review takes 1–3 days; web is instant).

7.3 When you get stuck

You will get stuck, and it will usually be environment problems (builds, certificates, store rejections) rather than code. In order: paste the full error message to your AI tool and ask it to explain before fixing; search the exact error text — Expo's forums and docs cover nearly everything; and if truly walled, hire an hour of help (an Expo developer on Upwork can often fix setup issues in a single session — one \$50 hour beats a lost week, and this is the best use of flexibility in your budget).

7.4 The budget

Item	Cost	When
Apple Developer Program	\$99/year	Needed before iOS store release (week ~8)
Google Play Console	\$25 one-time	Same timing, Android
Domain name	~\$12/year	Week 1
Cursor (AI editor)	\$20/month	From day one — this is your main tool
Supabase, Vercel, Expo, RevenueCat, PostHog	\$0	Free tiers cover launch scale
Occasional expert hour	~\$50 as needed	When stuck > 2 days

Steady state lands at roughly \$30–35/month plus the annual fees — inside your budget, with slack for an expert hour when needed.

RECOMMENDED ACTION: Today: install Cursor, create a GitHub account, and ask the AI to scaffold a new Expo project and migrate your v5 file into the Section 2.2 structure. That first working migration teaches you 80% of the workflow.

8. The 16-Week Roadmap

Phase	Weeks	What ships
Foundation	1–2	Expo project scaffolded; v5 content split into JSON; screens into files; app runs on your phone via Expo Go.
Core build	3–6	First-session flow (Section 3.2), accounts + progress sync (Supabase), streaks, polished lessons 1–4, dark mode.
Monetization + web	7–8	RevenueCat + paywall + tiers wired; analytics events (Section 9.2); web version deployed; 10 launch clips recorded.
Web launch	9	Product Hunt + Show HN + Reddit. Watch funnels daily; fix what real users trip over.
Store prep	10–12	Fixes from web feedback; store listings, screenshots, privacy labels; submit to both stores; apply to small-business fee programs.
Store launch	13	Apps live. Rating prompts on. Social clips continue at 3/week.
First audit cycle	14–16	First monthly blindspot audit (Section 9.3); first pricing-page funnel review; decide next quarter from data, not vibes.

Sixteen weeks is deliberately unhurried. First-time builders who rush to the store on week 6 launch a fragile app to their most important early audience. The web-first sequence gives you a real audience and real bug reports while mistakes are still cheap.

RECOMMENDED ACTION: Put the week-9 web launch date in your calendar now and treat it as fixed. Scope flexes; the date does not. A shipped smaller app beats an unshipped complete one.

9. The Self-Refuting, Self-Improving System

You asked for a project that refutes and improves itself, and for a way to stay out of ambiguity. Here is that system, concretely. The core idea: every significant belief about the product must be written down as a claim that a number can prove wrong, and you must schedule the moments when you check. Solo founders fail less from bad ideas than from unexamined ones — nobody is around to disagree with you, so the process has to do it.

9.1 Falsifiable claims

Before building any significant feature, write one sentence in a decision log (a simple document): what you believe, the number that would prove it wrong, and the date you will check. Examples for this app: 'Users want guided lessons — refuted if fewer than 40% of installers finish lesson 1 in month one.' 'The lesson-4 paywall is at the right spot — refuted if fewer than 5% of users who hit it start a trial within a week.' 'Short-form clips drive installs — refuted if 30 posted clips produce fewer than 200 store visits.' When a claim is refuted, the response is a change to the product or plan — not a softer restatement of the claim. That is what makes the process self-refuting rather than self-justifying.

9.2 Instrument on day one

None of this works without measurement, which is why analytics goes in during week 7, not after launch. The minimum event set: app opened, lesson started, lesson completed (with duration), quiz taken (with score), paywall viewed, trial started, subscribed, cancelled, ad watched (later). With those nine events, PostHog can answer nearly every question in this section. If you cannot name the event that would refute a feature, you do not understand the feature yet — which is itself useful information.

9.3 The monthly blindspot audit

One hour, first Saturday of each month, five standing questions. One: what number did I not look at this month because I feared what it would say? Look now. Two: what has stayed in the plan only because removing it feels like wasted work? (Sunk-cost check — the kids section is a likely early candidate.) Three: what did the last three users I actually talked to say, and if the answer is 'I have not talked to any users,' that is the finding — fix it before next month. Four: which claim in the decision log is past its check date? Check it. Five: what would a skeptical friend say is obviously wrong right now? Write the answer even when it stings. Then update Section 10's register: retire resolved blindspots, add new ones. The register is a living document; this plan is version 1 of it.

9.4 Kill and pivot criteria, written while calm

Decide the thresholds now, before emotion is involved. Suggested: if at six months post-store-launch the app has under 300 monthly active users and under \$100/month revenue despite consistent content marketing, stop building features and spend one full month only on distribution experiments. If at twelve months revenue is under \$300/month, the venture as a business is refuted — then make the deliberate choice to continue as a hobby, pivot the asset (the animation engine could serve corporate financial-literacy training, a genuinely underserved buyer with budgets), or wind down cleanly. Writing these numbers down today is what makes the eventual decision honest.

9.5 Escaping ambiguity

When you feel lost — and you will — it is nearly always because a question is unresolved, not because the work is hard. The protocol: write the question in one sentence; if data can answer it, pull the data; if a user can answer it, ask three users; if only building can answer it, build the smallest possible test of it

this week. Never sit in ambiguity for more than a week — convert it into a question, then into an action. Ambiguity is a queue of unasked questions, nothing more.

RECOMMENDED ACTION: Create the decision log today (a plain Google Doc titled 'Claims'), write your first five falsifiable claims, and put a repeating monthly 'Blindspot audit — 1 hour' event in your calendar before doing anything else in this plan.

10. Blindspot Register, Version 1

These are the current known blindspots — things not on your list that can hurt this project. Each audit updates this register.

10.1 Legal: this app is adjacent to financial advice

Your lessons include 'Best Investments' and 'Avoid' per cycle phase. Education about asset classes is legal; personalized investment advice is regulated. Keep everything general and historical ('equities have historically performed well early in expansions'), never personal ('you should buy...'). Add a plain-language disclaimer on first launch and in settings: educational content, not investment advice, no guarantee of accuracy. This is not a lawyer's opinion — a one-hour consult before store launch (~\$200–300, a worthwhile one-time exception to the budget) is cheap insurance.

10.2 The Dalio dependency

The prototype credits Ray Dalio's framework and quotes him. Teaching ideas is fine; building a brand on his name and direct quotes without permission is risk — both legal (right of publicity, quoted text) and platform (store takedown on complaint). Reframe as 'inspired by principles popularized by economists and investors,' remove direct quotes from the app and all marketing, and let the credit live in an acknowledgments line.

10.3 Kids content changes your legal category

A kids section aimed at ages 5–17 can make the app 'child-directed' under COPPA (US) and similar laws elsewhere. That restricts data collection, changes app-store privacy classification, and severely restricts ads — mixing AdMob with child-directed content is a well-known way for small apps to get removed. Simplest resolution for v1: ship the kids content as a parent-facing feature ('teach your kids'), inside Premium, with no ads anywhere near it and no accounts for children. Revisit a dedicated kids mode only if data shows real demand.

10.4 Five languages is a maintenance debt

Every content change now requires five translations, and machine-translated financial content in a paid product reads as cheap to native speakers. Resolution: English is the product; the other four are beta until each is human-reviewed (Korean presumably first, by you). Do not market in a language you cannot support user emails in.

10.5 Solo-founder single points of failure

If your laptop dies, is the project safe? GitHub solves code; also export Supabase data monthly and keep store credentials and 2FA recovery codes in a password manager. If you stop for a month, does the product decay? Design content marketing so clips can be batch-produced ahead. These are boring risks, which is exactly why they belong in the register.

10.6 The quiet failure mode: building instead of distributing

The most likely failure is not a crash or a rejection — it is six months of pleasant feature-building with no users, because building feels productive and distribution feels uncomfortable. The register exists partly to name this: after week 9, if any month shows zero distribution experiments, that is a red-flag audit finding regardless of how much code shipped. A useful standing rule after launch: half your weekly hours go to distribution until 1,000 MAU.

RECOMMENDED ACTION: Resolve 10.1–10.3 in week 1 — all three are content edits that take hours now and become expensive after launch: soften investment language and add the disclaimer, strip Dalio quotes, move kids content behind the parent framing.

11. Your First Five Moves

Everything above compresses to this. First, create the decision log and calendar the monthly audit (Section 9 — thirty minutes). Second, make the three content edits from the blindspot register (Section 10 — a few hours). Third, install Cursor, create GitHub and Expo accounts, and migrate the v5 prototype into the new structure (Section 2.2 — your first two weeks, and your apprenticeship in the workflow). Fourth, build the first-session flow end-to-end before anything else (Section 3.2). Fifth, put the week-9 web launch date in your calendar and protect it. When those five are done, this document's roadmap takes over — and in month four, the audit process starts telling you what this plan got wrong, which is exactly what it is designed to do.